

Formal Modelling of an Ephemeral Trust for Secure CI/CD Runners

Paulo Duarte¹[0000–0002–9976–5729], Simão Melo de Sousa²[D719–2946–CFE8],
and Joel Guerreiro³[0000–0001–7471–4928]

¹ NOVA LINCS, University of Algarve, Faro, Portugal, prduarte@ualg.pt

² NOVA LINCS, University of Algarve, Faro, Portugal, spsousa@ualg.pt

³ NOVA LINCS, University of Algarve, Faro, Portugal, jdguerreiro@ualg.pt

Abstract. Ephemeral CI/CD runners have emerged as a key mechanism for reducing software supply-chain risk, yet trusted execution is commonly treated as a static infrastructure property established before execution.

This paper presents a formally verified lifecycle-oriented model for hardened ephemeral CI/CD runner execution. The proposed model treats trusted execution as a dynamic authorization process rather than a static infrastructure property, combining attestation-bound trust establishment, immutable policy enforcement, ephemeral secret leasing, consumable execution capabilities, and fail-closed revocation semantics.

The system is specified in TLA+ as a state-transition model that captures the complete runner lifecycle, from provisioning and attestation through authorization, execution, revocation, and teardown. To evaluate its resilience under hostile conditions, the specification explicitly incorporates adversarial transitions representing evidence replay, session replay, unauthorized policy injection, cross-runner secret reuse, and use-after-expiry execution attempts. Correctness is established through exhaustive TLC model checking against 38 safety properties and 9 liveness properties governing authorization freshness, trust consistency, execution isolation, secret lifecycle management, immutable policy enforcement, and eventual revocation.

Exhaustive results confirms that the modeled adversarial behaviors cannot establish or preserve unauthorized trusted execution, violate policy integrity, or maintain execution capability beyond its authorized lifetime. This work establishes a formally verified foundation for secure ephemeral CI/CD runners, demonstrating how lifecycle-oriented authorization semantics can provide strong guarantees for replay resistance, temporal isolation, revocation correctness, and fail-closed execution in zero-trust software delivery infrastructures.

Keywords: Secure Runners · CI/CD Security · Formal Verification · TLA+ · Zero Trust · Supply Chain Security

1 Introduction

Continuous Integration and Continuous Deployment (CI/CD) infrastructures have evolved from software automation tools into security-critical components of modern software supply chains. Contemporary CI/CD platforms routinely process highly sensitive assets, including source code, cryptographic signing material, deployment credentials, software dependencies, and release artifacts. Consequently, compromising the execution infrastructure of a CI/CD pipeline may directly affect the integrity and trustworthiness of downstream software distributed to thousands or even millions of users [8].

Several high-profile software supply-chain incidents, including SolarWinds, Codecov, and the 3CX compromise, have demonstrated that attackers increasingly target build infrastructures rather than the deployed applications themselves [12, 9, 20, 30]. These incidents have motivated substantial research and industrial efforts toward strengthening software supply-chain security through provenance frameworks, artifact traceability, dependency verification, and hardened build processes. Frameworks such as SLSA and in-toto significantly improve software provenance by introducing reproducible build requirements, signed metadata, and tamper-evident workflow descriptions [28, 38].

Despite these advances, the runtime execution environment itself often remains insufficiently modeled from a security perspective. CI/CD runners are frequently trusted after an initial authorization decision, subsequently retaining access to deployment credentials, secret material, artifact publication privileges, and unrestricted network communication throughout execution. Although implementation-level hardening mechanisms exist, authorization validity is rarely treated as a continuously evolving property whose correctness depends on the current execution state, attestation freshness, policy consistency, secret validity, and revocation status [6, 36, 37].

This limitation becomes particularly significant in cloud-native environments where runners are provisioned dynamically, execute short-lived workloads, and are destroyed immediately after completion. In such environments, trust should no longer be interpreted as a static property assigned to infrastructure, but rather as a temporary authorization state whose validity must evolve consistently throughout the complete execution lifecycle. This perspective aligns naturally with Zero Trust principles, which require explicit verification, least privilege, continuous authorization assessment, and rapid privilege revocation whenever trust assumptions change [32].

Existing CI/CD security mechanisms primarily strengthen software provenance, workload identity, and infrastructure hardening, but they provide limited support for reasoning about whether authorization remains valid throughout execution under adversarial conditions.

This paper addresses this problem by presenting a formally verified lifecycle-oriented model for hardened ephemeral CI/CD runners. The proposed model combines attestation-bound trust establishment, immutable policy enforcement, ephemeral secret leasing, consumable execution capabilities, and fail-closed revocation semantics within a unified authorization lifecycle. Rather than modeling

trust as a single authorization event, the specification captures how authorization evolves from provisioning and attestation through execution, revocation, and teardown.

The complete lifecycle is formalized in TLA+ as a state-transition system. In addition to trusted operational behavior, the model explicitly incorporates adversarial transitions representing evidence replay, session replay, unauthorized policy injection, cross-runner secret reuse, and use-after-expiry execution attempts. Correctness is established through exhaustive TLC model checking covering authorization freshness, trust consistency, execution isolation, immutable policy enforcement, secret lifecycle management, and eventual revocation.

The main contributions of this work are as follows:

1. We present a lifecycle-oriented formal model for ephemeral CI/CD runner execution in which trust establishment, authorization, secret leasing, policy enforcement, execution capability, and revocation are represented as explicit state transitions.
2. We integrate representative adversarial behaviors directly into the transition system, enabling formal verification against replay attacks, stale authorization reuse, unauthorized policy modification, cross-runner secret reuse, and fail-closed revocation scenarios.
3. We formally verify the proposed lifecycle model using TLA+ and exhaustive TLC model checking, establishing 38 safety properties and 9 liveness properties under both trusted and adversarial execution scenarios.
4. We release the complete formal specification, verification configurations, execution logs, and reproducibility artifacts as open-source research artifacts to facilitate independent validation and future extensions.

2 Background

CI/CD Runner Security and Ephemeral Execution. Modern Continuous Integration and Continuous Deployment (CI/CD) platforms rely on runner infrastructures responsible for executing workflow tasks such as source-code retrieval, dependency resolution, compilation, testing, artifact generation, signing, and deployment orchestration [17, 16]. Because runners routinely access highly privileged assets, including deployment credentials, cryptographic signing keys, software artifacts, and proprietary source code, they represent one of the most security-sensitive components of contemporary software supply chains.

To reduce long-lived attack surfaces, modern CI/CD platforms increasingly adopt ephemeral execution models in which isolated runner instances are provisioned for a single workload and destroyed immediately after execution completes. This approach minimizes residual state, reduces cross-workload contamination, and limits the persistence of compromised execution environments. Ephemeral runners also align naturally with Zero Trust principles by promoting short-lived identities, explicit authorization, least privilege, and rapid privilege revocation [32].

Nevertheless, ephemeral execution alone does not guarantee secure execution semantics. If authorization remains valid after replay, secret leases outlive their intended execution window, or communication policies can be modified during execution, ephemeral infrastructure may still be vulnerable to replay attacks, stale authorization reuse, privilege persistence, and policy manipulation. Consequently, protecting the runner lifecycle requires not only isolated execution environments but also formally defined authorization semantics governing trust establishment, execution, revocation, and teardown.

Trusted Execution and Remote Attestation. Trusted Execution Environments (TEEs) and confidential computing technologies provide hardware-assisted mechanisms for protecting workload integrity and confidentiality within partially trusted infrastructures [34]. By isolating execution from privileged software layers, TEEs reduce the trusted computing base and enable stronger guarantees regarding runtime integrity.

Current hardware platforms adopt different isolation models. Intel SGX provides enclave-based protection for selected application components together with cryptographic attestation mechanisms capable of proving enclave identity and integrity [24, 11]. AMD SEV-SNP and Intel TDX extend hardware-assisted isolation to complete virtual machines, protecting larger execution environments while supporting remote attestation of virtualized workloads [21, 1, 19]. These technologies increasingly form the foundation of confidential computing deployments within cloud infrastructures.

Within secure CI/CD environments, remote attestation enables external authorities to validate runner integrity before releasing secrets or authorizing privileged operations. However, attestation alone establishes only the initial trust decision. It does not define how authorization should evolve throughout execution, how replay attempts should be handled, or when execution privileges must be revoked. These lifecycle concerns motivate the authorization model proposed in this work.

Formal Verification with TLA+. Formal verification provides mathematically rigorous techniques for establishing whether a system satisfies specified correctness properties under every reachable execution scenario. Rather than validating individual execution traces through testing, formal methods reason over all behaviors permitted by a formal specification [22].

TLA+ models distributed systems as state-transition systems whose behavior evolves through predicates relating successive global states. Safety properties specify conditions that must never be violated, whereas liveness properties express progress requirements ensuring that desirable events eventually occur under appropriate fairness assumptions [22]. The TLC model checker exhaustively explores the reachable state space and determines whether these properties hold across every execution trace admitted by the specification [26].

These characteristics make TLA+ particularly well suited to lifecycle-oriented security analysis. Authorization evolution, replay resistance, trust establishment, policy immutability, revocation semantics, and adversarial behavior can all be expressed directly as state transitions and verified exhaustively over the complete

execution lifecycle. This capability provides stronger assurance than reasoning solely about isolated protocol steps or implementation-specific execution traces and forms the basis of the formal model presented in this paper.

3 Threat Model and Security Objectives

This section defines the assumptions under which the proposed model operates, the capabilities attributed to the adversary, and the security objectives that guide both the architectural design and the formal verification.

3.1 System Assumptions

The proposed model considers a hardened CI/CD architecture composed of ephemeral runner instances, a CI/CD orchestration service, an attestation authority, a policy authority, a secret management service, and external communication endpoints. Each CI/CD job executes within a dedicated ephemeral runner provisioned exclusively for a single workload and destroyed immediately after execution completes or security violations are detected.

The Trusted Computing Base (TCB) consists of the orchestration service, attestation authority, policy authority, and secret management infrastructure. The model further assumes the correctness of the employed cryptographic primitives, authenticated communication between trusted components, secure nonce generation, and the integrity of the underlying attestation mechanism.

Conversely, runner instances, executed workloads, third-party dependencies, runtime artifacts, and external communication channels are treated as potentially hostile. The proposed model therefore assumes that security cannot rely on runner integrity alone and must instead be enforced through explicit authorization semantics governing the complete execution lifecycle.

3.2 Adversary Model

The adversary is assumed to possess substantial capabilities within the execution environment while remaining unable to compromise the trusted control plane. More specifically, an attacker may capture previously observed attestation evidence, replay execution sessions, attempt unauthorized policy modification, reuse secret material across independent runners, and exploit delayed revocation or authorization expiration throughout the runner lifecycle.

Accordingly, the model explicitly incorporates the following adversarial behaviors:

- Replay of previously observed attestation evidence.
- Replay of stale execution sessions.
- Unauthorized modification of communication policies.
- Cross-runner reuse of secret material.
- Use-after-expiry attempts targeting execution capabilities or secret leases.

The model intentionally excludes compromise of trusted authorities, hardware attacks against TEEs, cryptographic breakage, implementation vulnerabilities within the attestation infrastructure, and denial-of-service attacks. These assumptions allow the formal verification to focus specifically on lifecycle-oriented authorization correctness rather than on implementation-specific failures.

3.3 Security Objectives

Based on the above assumptions, the proposed model aims to preserve authorization correctness throughout the complete execution lifecycle. The following objectives directly motivate the formal safety and liveness properties introduced in Section 5.

- **O1 — Attestation-Bound Trust.** Trust establishment requires successful attestation together with explicit policy authorization bound to both the originating runner identity and its associated execution session.
- **O2 — Ephemeral Authorization.** Execution capability is modeled as a temporary, single-use authorization that cannot be reused once consumed and remains subject to explicit revocation.
- **O3 — Ephemeral Secret Leasing.** Secret material is released only to trusted execution contexts and remains valid exclusively for the corresponding authorized execution lifecycle.
- **O4 — Immutable Policy Enforcement.** Communication policies become immutable after authorization, preventing runtime privilege expansion or unauthorized network access.
- **O5 — Fail-Closed Revocation.** Replay detection, attestation failure, authorization expiration, or policy violations immediately invalidate execution capability, revoke active secret leases, and terminate trusted execution.
- **O6 — Replay and Reuse Resistance.** Previously observed attestation evidence, stale sessions, expired secret leases, or residual authorization state must never establish or preserve valid trusted execution.

Collectively, these objectives define the security requirements that drive both the architecture and the formally verified properties presented throughout the remainder of this paper.

4 System Architecture

This section presents the reference architecture underlying the proposed formal model. The architecture follows a Zero Trust design philosophy in which execution privileges are never assumed to persist throughout the lifetime of a CI/CD runner. Instead, authorization is continuously constrained by attestation validity, policy consistency, secret availability, and explicit revocation semantics.

Unlike conventional CI/CD infrastructures, where runners often retain privileges once execution begins, the proposed architecture models trusted execution

Table 1. Relationship between the security objectives and the formally verified properties.

Objective	Security Goal	Representative Verified Properties
O1	Attestation-bound trust establishment	<i>Inv_Freshness</i> , <i>Inv_TrustConsistent</i> , <i>Inv_NoTrustFromReplayedEvidence</i>
O2	Single-use, revocable execution authorization	<i>Inv_ExecutionAuthorized</i> , <i>Inv_ExecutionCapabilityConsumed</i> , <i>Inv_RevokedState</i>
O3	Ephemeral secret leasing	<i>Inv_SecretAccessValid</i> , <i>Inv_NoCrossRunnerSecretReuse</i> , <i>Inv_SecretLeaseBound</i>
O4	Immutable policy enforcement	<i>Inv_PolicyLockedImmutable</i> , <i>Inv_NoInjectedEgressAllowed</i>
O5	Fail-closed revocation	<i>Inv_TrustDestroyed</i> , <i>Inv_RevokedState</i> , <i>SessionReplayEventuallyTerminated</i>
O6	Replay and reuse resistance	<i>Inv_NoTrustFromReplayedEvidence</i> , <i>Inv_NoTrustFromReplayedSession</i> , <i>Inv_NoUseAfterTrustExpiry</i>

as a sequence of explicitly controlled lifecycle phases. Provisioning, attestation, authorization, secret distribution, execution, revocation, and teardown are coordinated by trusted control-plane services, while workload execution remains confined to isolated ephemeral runners.

Figure 1 presents the overall architecture of the proposed Secure Runner.

4.1 Architectural Components

The architecture comprises five principal architectural components.

CI/CD Orchestrator. The orchestration service is responsible for provisioning ephemeral runner instances, coordinating their lifecycle, initiating attestation, requesting policy evaluation, and triggering teardown after execution completes or security violations are detected. The orchestrator never grants execution privileges directly, but instead coordinates interactions among the trusted control-plane services.

Attestation Authority. The attestation authority validates the integrity of newly provisioned runners before privileged operations become possible. Successful attestation establishes confidence that the execution environment satisfies the required integrity assumptions and provides the basis for subsequent authorization decisions.

Policy Authority. The policy authority evaluates whether the attested runner is permitted to execute the requested workload. In addition to determining execution eligibility, it generates the immutable communication policy that constrains the runner throughout its execution lifetime.

Secret Management Service. Sensitive execution material is distributed exclusively through the secret management service. Rather than issuing persis-

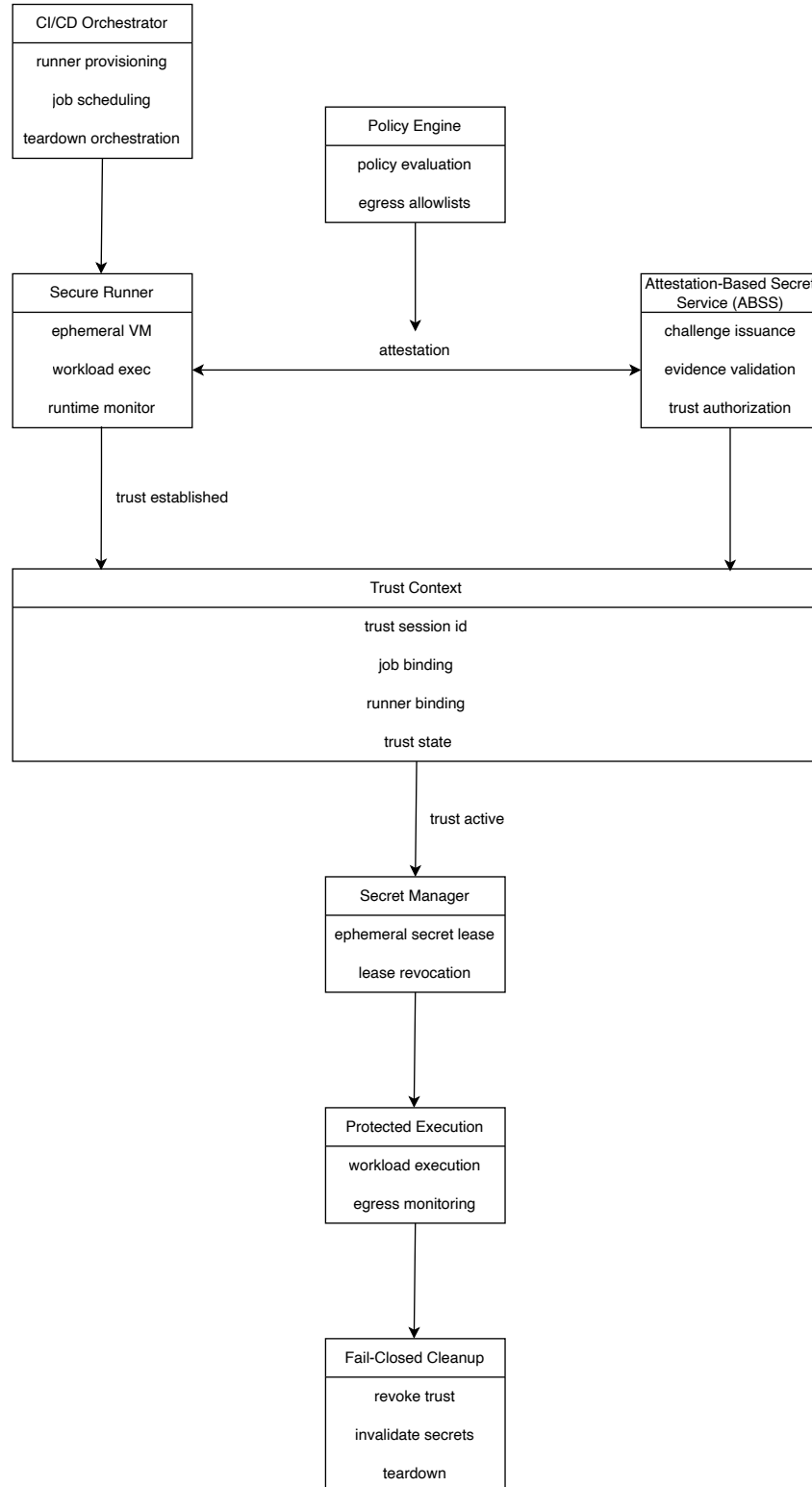


Fig. 1. Reference architecture of the hardened ephemeral CI/CD runner model.

tent credentials, secrets are released as ephemeral leases bound to the authorized runner identity and execution session. Secret validity therefore depends on the continued validity of the corresponding trust relationship.

Ephemeral Runner. Each workload executes within an isolated runner provisioned exclusively for a single CI/CD job. Newly created runners possess no trust relationship, execution capability, secret access, or communication privileges. All privileged state is acquired dynamically after successful attestation and authorization and destroyed during teardown.

External communication endpoints remain outside the trusted computing base and are accessible only through explicitly authorized communication policies.

4.2 Trust Boundaries

The architecture explicitly separates the trusted control plane from the untrusted execution plane.

Trusted components, including the orchestrator, attestation authority, policy authority, and secret management service, are responsible for establishing and maintaining authorization correctness. In contrast, runner instances, workloads, software dependencies, runtime artifacts, and external communication channels are treated as potentially adversarial throughout execution.

This separation minimizes the trusted computing base while ensuring that execution privileges depend exclusively on externally verified authorization state rather than on assumptions regarding runner integrity.

4.3 Execution Lifecycle

The execution lifecycle begins when the orchestration service provisions a new ephemeral runner. During bootstrap, the runner remains untrusted and operates without execution capability, secret access, or authorized communication privileges.

Successful attestation establishes the integrity of the execution environment, after which the policy authority determines whether execution is permitted. Only when both conditions are satisfied does the runner transition into a trusted execution state eligible to receive ephemeral secret leases and a single-use execution capability bound to the corresponding runner identity and execution session.

Execution authorization is intentionally consumable. Once execution begins, the capability cannot be reused for subsequent workloads or replayed sessions. Throughout execution, communication remains constrained by the immutable policy established during authorization, while secret leases remain valid only for the corresponding trusted execution context.

Replay detection, policy violations, attestation failure, authorization expiration, or stale session reuse immediately trigger fail-closed revocation semantics. Execution capability, trust bindings, secret leases, and session associations are destroyed before the runner is terminated and permanently removed.

Figure 2 summarizes the resulting lifecycle, which forms the conceptual basis for the formal state-transition model introduced in Section 5.

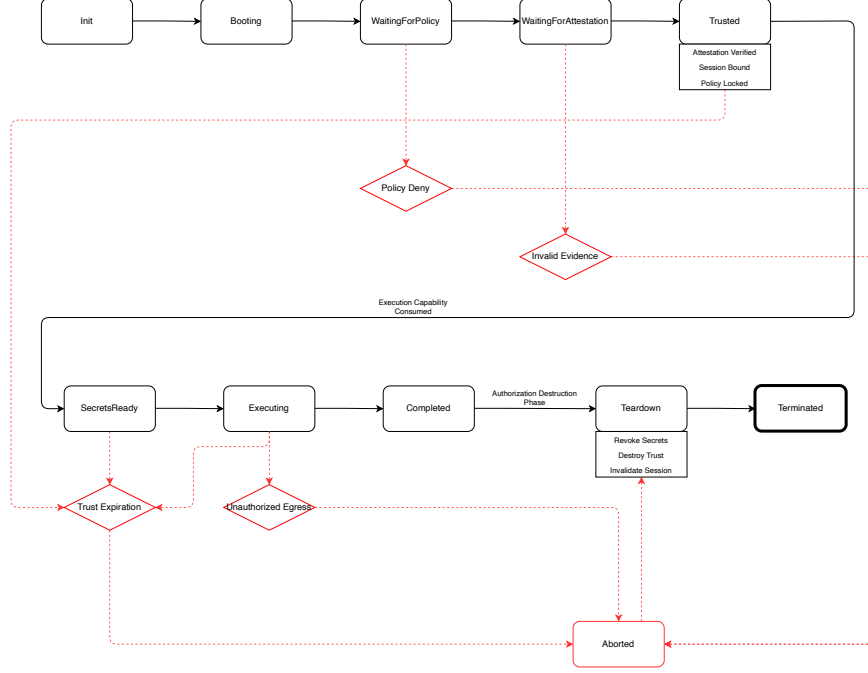


Fig. 2. Lifecycle-oriented state transitions of hardened runner execution.

5 Formal Model

This section presents the formal specification of the proposed Secure Runner architecture. The objective of the model is not to reproduce implementation details, but to provide a mathematically precise description of the security semantics governing ephemeral CI/CD runner execution. The specification captures how trust is established, how execution authorization evolves over time, how secrets are managed, and how revocation propagates throughout the execution lifecycle under both benign and adversarial conditions.

The specification is written in TLA+, which models distributed systems as state-transition systems over a global state. This formalism is particularly well suited to security-critical infrastructures because it enables reasoning about the temporal evolution of authorization rather than individual execution events. Instead of verifying isolated protocol steps, the model captures complete execution

traces spanning provisioning, attestation, authorization, execution, revocation, and teardown.

Unlike conventional access-control models, which frequently represent authorization as a static decision evaluated once before execution begins, the proposed specification treats authorization as a dynamic lifecycle property. Trust establishment, execution capability, secret validity, communication policies, and revocation state evolve independently but remain mutually constrained throughout execution. Consequently, authorization correctness depends not only on the current system state but also on the sequence of transitions that produced it.

To enable exhaustive verification, the specification explicitly incorporates adversarial behavior into the transition relation. Replay attacks, stale authorization reuse, unauthorized policy mutation, cross-runner secret reuse, and use-after-expiry execution attempts are therefore explored by the model checker together with legitimate system evolution. This unified transition model allows security guarantees to be established over every reachable execution satisfying the assumptions defined in Section 3.

Table 2 summarizes the overall size and structure of the formal specification.

Table 2. Summary of the proposed formal model.

Metric	Value	Metric	Value	Metric	Value
TLA+ modules	3	Trusted transitions	14	Safety properties	38
Specification LOC	1487	Lifecycle transitions	5	Liveness properties	9
State variables	30	Adversarial transitions	7	Total transitions	26

5.1 Modeling Assumptions

As with any formal specification, the proposed model intentionally abstracts away implementation-specific details in order to focus on the security properties of interest. The objective is to preserve the semantics governing authorization correctness while maintaining a finite state space suitable for exhaustive exploration.

The specification therefore does not model operating-system internals, virtualization mechanisms, network protocols, cryptographic algorithms, or hardware implementations of trusted execution environments. Instead, these mechanisms are represented through abstract predicates and state variables capturing only their observable security effects. For example, remote attestation is modeled as the establishment of a valid trust relationship rather than through the underlying cryptographic protocol, while secret distribution is represented by explicit lease state instead of concrete key-management operations.

Similarly, CI/CD jobs are treated as abstract workloads whose internal computation is irrelevant to the security properties being verified. The model therefore focuses exclusively on authorization evolution, trust establishment, execu-

tion capability ownership, policy enforcement, secret lifecycle management, and revocation semantics.

These abstractions allow the specification to isolate the correctness of lifecycle-oriented authorization independently of implementation details while remaining sufficiently expressive to model the adversarial behaviors introduced in Section 3.

5.2 State Representation

The global state of the specification captures the security-relevant information required to reason about authorization correctness throughout the execution lifecycle. Rather than modeling the internal implementation of individual system components, the specification represents the observable security state associated with each runner together with the shared state maintained by the trusted control plane.

Each runner evolves independently through a collection of state variables describing its lifecycle, trust relationship, execution authorization, secret availability, communication policy, and revocation status. Global variables represent information shared across the system, including policy decisions, active sessions, and adversarial observations. This organization enables multiple runner instances to execute concurrently while preserving explicit isolation between independent execution contexts.

To maintain a clear separation of concerns, the state variables are organized into six logical categories, summarized in Table 3. Although each variable evolves independently through the transition relation, the model constrains their evolution through formally verified invariants that preserve authorization correctness across all reachable executions.

Table 3. Logical organization of the specification state.

State Category	Purpose and Representative Variables
Runner Lifecycle	Represents the execution state and ownership of each ephemeral runner (<code>runnerState</code> , <code>runnerOwner</code>).
Trust Management	Captures attestation status and trust bindings established between runners and execution sessions (<code>trustState</code> , <code>trustSessionId</code> , <code>attestationValid</code>).
Execution Authorization	Represents authorization decisions and consumable execution capabilities (<code>executionCapability</code> , <code>policyDecision</code>).
Secret Management	Models ephemeral secret leases and secret availability during authorized execution (<code>secretLeaseState</code> , <code>secretsGranted</code>).
Communication Policy	Represents immutable egress policies governing authorized communication (<code>allowedEgress</code>).
Adversarial State	Tracks replay attempts and other hostile observations introduced by adversarial transitions (<code>replayedEvidence</code> , <code>replayedSession</code>).

This logical decomposition deliberately avoids representing authorization as a single Boolean state. Instead, trusted execution emerges from the consistent interaction of multiple state variables describing trust establishment, capability ownership, secret validity, and policy authorization. This separation enables the specification to distinguish between states that would appear equivalent under simpler authorization models—for example, a runner that has successfully completed attestation but has not yet received execution capability, or a runner whose trust relationship remains valid after its secret lease has expired.

Modeling authorization as the coordinated evolution of multiple security states also enables adversarial behaviors to be analyzed with greater precision. Replay attacks, stale authorization reuse, policy mutation, and secret leakage affect different portions of the global state, allowing the specification to verify that violations affecting one security dimension cannot silently propagate to the remaining authorization state.

5.3 Transition System

The behavior of the Secure Runner architecture is specified through an explicit state-transition system consisting of an initialization predicate (**Init**) and a transition relation (**Next**). The initialization predicate defines the security state from which every execution begins, while the transition relation captures every permissible evolution of the system throughout the runner lifecycle.

Initially, every runner is provisioned in an untrusted state without execution capability, secret leases, or active trust relationships. Consequently, execution cannot begin until the complete authorization workflow—including attestation, policy validation, and secret distribution—has been successfully completed.

Only representative excerpts are shown below. The complete specification is publicly available in the accompanying repository described in Section 8.

```
Init ==
  /\ ...
  /\ runnerState = [r \in RunnerIds |-> "Init"]
  /\ trustState = [r \in RunnerIds |-> "None"]
  /\ executionCapability = [r \in RunnerIds |-> FALSE]
  /\ secretLeaseState = [r \in RunnerIds |-> "None"]
```

```
Next == TrustedNext \/ LifecycleNext \/ AdversaryNext
```

Unlike sequential execution models, TLA+ represents system evolution as a nondeterministic choice between all enabled transitions. Consequently, every execution explored by TLC corresponds to one possible interleaving of trusted operations, lifecycle evolution, and adversarial behavior. This execution model naturally captures the asynchronous nature of distributed CI/CD infrastructures, where multiple runners evolve concurrently and security-relevant events may occur in different orders.

The transition relation is intentionally decomposed into three complementary categories:

- **Trusted transitions** represent legitimate control-plane operations, including runner provisioning, attestation, trust establishment, secret release, execution authorization, and normal workload execution.
- **Lifecycle transitions** capture the natural evolution of authorized execution, including capability consumption, lease expiration, trust revocation, execution termination, and runner teardown.
- **Adversarial transitions** explicitly model hostile behaviors such as replay attacks, stale session reuse, unauthorized policy mutation, cross-runner secret reuse, and use-after-expiry execution attempts.

This decomposition provides two important advantages. First, it separates normal system behavior from adversarial behavior without requiring multiple specifications or specialized attacker models. Second, it allows the model checker to explore arbitrary interleavings between trusted and hostile transitions, ensuring that the verified properties remain valid regardless of when adversarial actions occur during execution.

Figure 2 illustrates the conceptual lifecycle represented by these transitions. Although the implementation contains 26 individual transition actions, they collectively describe a single authorization lifecycle progressing from provisioning to trusted execution and eventually to revocation or successful teardown. The formal specification therefore captures both normal execution and failure scenarios within a unified transition system.

This transition-based representation also provides the foundation for the verification strategy adopted throughout the remainder of this work. Safety properties constrain the set of reachable states produced by the transition relation, while liveness properties constrain the long-term evolution of executions, ensuring that replay attempts, stale authorization, policy violations, and trust expiration cannot indefinitely preserve execution capability.

5.4 Authorization Semantics

The central design principle of the proposed specification is that execution authorization is not represented as a persistent privilege or a single Boolean state. Instead, authorization is modeled as a derived security condition emerging from the consistent interaction of multiple independent state variables.

This design reflects the observation that trusted execution within a Secure Runner depends simultaneously on several security guarantees. Successful attestation alone does not authorize execution, nor does the existence of valid secret leases or policy approval in isolation. Execution becomes permissible only when all required security conditions remain simultaneously satisfied throughout the execution lifecycle.

Accordingly, execution eligibility is defined through the derived predicate `ExecutionAuthorized`:

```
ExecutionAuthorized(r) ==
    executionCapability[r] /\ LeaseIssued(r) /\ IsTrustBound(r)
```

Unlike traditional authorization models in which execution permission is stored explicitly, the proposed formulation derives authorization from the simultaneous satisfaction of multiple security predicates. This design minimizes redundant state while making authorization invalidation implicit whenever any underlying security condition ceases to hold.

The execution capability represents the explicit authorization issued by the trusted control plane following successful policy evaluation. Secret leases represent the availability of temporary execution credentials whose lifetime remains bounded by the corresponding authorization context. Finally, trust binding guarantees that both authorization and secrets remain associated with the runner and execution session that successfully completed attestation.

Consequently, authorization is not stored explicitly as part of the global system state. Instead, it is derived from the consistency of several independently evolving security conditions. This prevents execution from remaining authorized when only part of the required security context is preserved. For example, an attested runner whose secret lease has expired, or a runner retaining execution capability after revocation, no longer satisfies the authorization predicate.

Modeling authorization in this manner provides stronger guarantees than conventional lifecycle models based on static authorization flags. Because capability ownership, trust establishment, and secret validity evolve independently, adversarial transitions affecting any one of these components immediately invalidate execution eligibility without requiring additional synchronization mechanisms.

This decomposition also simplifies formal verification. Rather than proving the correctness of a single authorization variable, the specification verifies the correctness of each underlying security condition independently together with the invariants governing their interaction. Consequently, authorization becomes an emergent property of the execution lifecycle rather than an explicitly stored system attribute, allowing revocation, replay resistance, and temporal consistency to arise naturally from the verified state-transition semantics.

Table 4 summarizes the lifecycle semantics of the authorization components modeled by the specification. It highlights that execution authorization is never represented explicitly as a persistent state, but instead emerges from the coordinated evolution of trust establishment, execution capability, secret leasing, and policy enforcement.

5.5 Verified Properties

Safety Safety properties establish that undesirable security states are unreachable throughout every execution explored by TLC. Within the proposed Secure Runner model, these properties guarantee that authorization correctness is preserved regardless of the ordering of trusted, lifecycle, or adversarial transitions. Consequently, every reachable system state satisfies the security objectives introduced in Section 3.

Rather than proving a single global invariant, the specification verifies 38 safety properties covering complementary aspects of the authorization lifecycle.

Table 4. Lifecycle semantics of the authorization components.

Component	Established by	Invalidated by
Trust Relationship	Successful attestation followed by EstablishTrust creates an active runner-bound trust relationship.	Attestation failure, evidence replay, session replay, trust expiration, policy violations, and teardown revoke or destroy trust.
Execution Capability	ReleaseSecrets issues a single-use execution capability after successful authorization.	Consumed by StartExecution or revoked by policy violations, trust expiration, session replay, secret misuse, and teardown.
Secret Lease	ReleaseSecrets creates an ephemeral lease bound to the authorized runner and trust context.	Consumed during execution or revoked after expiration, policy violations, session replay, cross-runner reuse, or teardown.
Communication Policy	Policy evaluation authorizes the allowed egress set, which becomes immutable after EstablishTrust .	Never modified after authorization; violations immediately trigger fail-closed revocation.
Execution Authorization	Derived from the simultaneous validity of trust binding, execution capability, and secret lease.	Automatically becomes false whenever any underlying security condition is invalidated.

Collectively, these invariants constrain trust establishment, execution capability ownership, secret lifecycle management, policy enforcement, runner isolation, revocation semantics, and adversarial resilience.

Table 5 summarizes the complete set of verified safety invariants according to the security concern they enforce. This organization reflects the modular structure of the specification while illustrating how multiple classes of invariants jointly preserve authorization correctness.

These categories collectively establish that trust relationships remain bound to the originating runner and execution session, replayed attestation evidence cannot establish trusted execution, execution capability remains single-use, secret leases remain valid only during authorized execution, communication policies remain immutable after authorization, revoked authorization cannot regain validity, and secrets or execution capability cannot propagate across independent runners.

As an illustrative example, the following invariant guarantees that replayed attestation evidence cannot activate a trusted execution state:

```

Inv_NoTrustFromReplayedEvidence ==
  \A r \in RunnerIds :
    replayedEvidence[r] => trustState[r] # "Active"

```

Table 5. Complete set of verified safety invariants.

Category	Invariants
Foundation	<i>TypeInvariant</i> , <i>Inv_UniqueJobAssignment</i> , <i>Inv_UniqueActiveSessions</i> , <i>Inv_UniqueActiveNonces</i>
Trust and Attestation	<i>Inv_ActiveTrustIsBound</i> , <i>Inv_UsedActiveTrustIsBound</i> , <i>Inv_NoExecBeforeTrust</i> , <i>Inv_NoSecretsBeforeTrust</i> , <i>Inv_Freshness</i> , <i>Inv_EvidenceRunnerBinding</i>
Secret Leasing and Capabilities	<i>Inv_ExecutionCapabilitySingleUse</i> , <i>Inv_ConsumedLeaseCannotExecuteAgain</i> , <i>Inv_NoSecretsWithRevokedLease</i> , <i>Inv_ExecutionRequiresConsumedLease</i> , <i>Inv_RevokedLeaseNoRecovery</i>
Policy and Egress Enforcement	<i>Inv_EgressAuthorized</i> , <i>Inv_BlockedEgressFailsClosed</i> , <i>Inv_NoEgressAmbiguity</i> , <i>Inv_NoPolicyMutationAfterTrust</i> , <i>Inv_PolicyLockedImmutable</i> , <i>Inv_PolicyLockImpliesEstablishedTrust</i>
Isolation Guarantees	<i>Inv_NoCrossRunnerTrustBinding</i> , <i>Inv_NoCrossRunnerSecretLeak</i> , <i>Inv_NoCrossRunnerSecretReuse</i>
Lifecycle and Fail-Closed Behavior	<i>Inv_TrustDestroyed</i> , <i>Inv_TerminatedIsQuiescent</i> , <i>Inv_AbortIsSafe</i> , <i>Inv_NoUseAfterTrustExpiry</i> , <i>Inv_RevokedTrustNeverReactivates</i> , <i>Inv_TerminatedNoCapabilities</i>
Adversarial Resilience	<i>Inv_NoTrustFromReplayedEvidence</i> , <i>Inv_NoTrustFromReplayedSession</i> , <i>Inv_NoInjectedEgressAllowed</i> , <i>Inv_ReplayEvidenceForcesRevocation</i> , <i>Inv_ReplayedSessionForcesAbort</i>
Hardened Execution Consistency	<i>Inv_ExecutingRunnerIsFullyConsistent</i> , <i>Inv_ExecutionRequiresHealthySecurity</i> , <i>Inv_TrustUseOnlyAfterExecutionStarted</i>

Unlike implementation-level testing, which evaluates only selected execution traces, exhaustive model checking verifies that these invariants hold across every reachable system state generated during state-space exploration. Consequently, the verified guarantees apply independently of the order in which trusted operations and adversarial transitions are interleaved.

Liveness While safety properties prevent the system from reaching undesirable states, liveness properties guarantee that authorized execution continues to make progress and that security-critical lifecycle events eventually occur. Within the proposed Secure Runner model, liveness therefore complements safety by ensuring that authorization cannot remain indefinitely in intermediate or inconsistent states.

The specification verifies nine liveness properties covering lifecycle progression, trust establishment, execution, revocation, secret destruction, and adversarial recovery. Collectively, these properties ensure that every execution eventually

converges toward a consistent lifecycle outcome rather than remaining indefinitely suspended because of stale authorization, replay attempts, or incomplete revocation.

Table 6 groups these temporal properties according to the aspect of lifecycle progression they guarantee.

Particular emphasis is placed on fail-closed behavior. Once replay attacks, policy violations, attestation failures, trust expiration, or stale authorization reuse are detected, the affected runner must eventually lose execution capability, revoke any active secret leases, destroy its trust relationship, and progress toward termination. Consequently, adversarial actions cannot indefinitely preserve privileged execution simply by preventing further lifecycle evolution.

For example, the following temporal property guarantees that replayed sessions eventually terminate:

```
SessionReplayEventuallyTerminated ==
  \A r \in RunnerIds :
    replayedSession[r] ~> runnerState[r] = "Terminated"
```

The temporal operator $\sim>$ ("leads to") expresses eventuality in TLA+. Informally, whenever the predicate on the left becomes true, the predicate on the right must eventually hold. Under the fairness assumptions adopted by the specification, this property guarantees that replay-derived execution cannot remain active indefinitely and must ultimately converge to a terminated state.

The verified liveness properties also ensure that revocation propagates completely throughout the authorization lifecycle. Execution capability, trust bindings, secret leases, and session state cannot remain indefinitely active after authorization has been invalidated, thereby preventing residual privileged state from surviving beyond its intended lifetime.

Together, the verified safety and liveness properties establish complementary correctness guarantees. Safety prevents unauthorized execution from becoming reachable, whereas liveness guarantees that the lifecycle continues to evolve toward secure completion. The combination provides formal assurance that authorization freshness, replay resistance, temporal isolation, policy immutability, and fail-closed revocation are preserved throughout every execution explored by TLC.

5.6 Discussion

The proposed formal model deliberately emphasizes authorization semantics rather than implementation mechanisms. By abstracting away platform-specific details while explicitly modeling the evolution of trust, execution capability, secret leases, communication policies, and revocation, the specification isolates the security properties that remain invariant across different Secure Runner implementations. Consequently, the verified guarantees are independent of particular operating systems, container runtimes, hypervisors, or confidential computing technologies.

Table 6. Verified liveness properties.

Category	Property
Lifecycle	<i>EventuallyTerminatesAfterAbort</i>
Lifecycle	<i>EventuallyTerminatesAfterCompleted</i>
Lifecycle	<i>ExecutionEventuallyEnds</i>
Trust	<i>SecretsEventuallyRevokedAfterCompletionOrAbort</i>
Trust	<i>TrustEventuallyDestroyedAfterAbort</i>
Trust	<i>TrustEventuallyDestroyedAfterCompleted</i>
Trust	<i>LeaseEventuallyClosedAfterCompletionOrAbort</i>
Adversarial	<i>ReplayEventuallyHandled</i>
Adversarial	<i>SessionReplayEventuallyTerminated</i>

A central design principle of the specification is the separation of security concerns into independently evolving state variables constrained through formally verified invariants. Instead of representing authorization as a single persistent privilege, the model derives execution eligibility from the simultaneous validity of trust establishment, execution capability, and secret availability. This decomposition enables the specification to capture subtle lifecycle inconsistencies that would remain invisible in simpler authorization models relying on a single authorization flag.

The transition-based organization of the specification also promotes extensibility. New lifecycle phases, security mechanisms, or adversarial behaviors can be introduced by extending the transition relation without fundamentally modifying the existing state representation or verification strategy. Similarly, additional safety invariants and temporal properties can be incorporated incrementally while preserving the overall structure of the model.

Although the current specification was verified using exhaustive explicit-state exploration with TLC, the underlying model is intentionally tool-independent. Future work will therefore complement the presented results with symbolic verification and deductive proofs. These complementary techniques will provide stronger assurance over larger state spaces while enabling formal reasoning about properties that are difficult to establish through explicit-state exploration alone.

More generally, this work demonstrates that authorization can be modeled as a temporal property of execution rather than as a static access-control decision. By capturing trust establishment, capability ownership, secret leasing, policy enforcement, and revocation within a unified state-transition system, the specification provides formal guarantees over the complete Secure Runner lifecycle instead of individual protocol steps.

6 Verification under Adversarial Execution

The formal model presented in the previous section establishes the correctness of the Secure Runner authorization lifecycle under both trusted and adversarial

execution. This section presents the verification of the proposed formal model under adversarial execution, analyzing how explicitly modeled hostile behavior interacts with the transition system and how the verified safety and liveness properties preserve the security objectives introduced in Section 3.

Unlike implementation-level security evaluations, which typically assess individual attack scenarios through penetration testing or experimental validation, the proposed approach incorporates adversarial behavior directly into the formal specification. Hostile actions therefore become first-class transitions within the system model and are explored exhaustively together with legitimate execution by the TLC model checker.

This verification strategy provides stronger assurance than reasoning about isolated attack traces. Rather than demonstrating that a predefined set of attacks fails in specific situations, exhaustive state-space exploration establishes that every reachable execution satisfying the assumptions of the model preserves the specified correctness properties, regardless of the ordering or interleaving of trusted and adversarial transitions.

The adversarial behaviors considered in this work correspond directly to the threat model defined in Section 3. They target the security properties most critical to hardened ephemeral CI/CD execution, namely authorization freshness, trust establishment, replay resistance, secret isolation, immutable policy enforcement, and fail-closed revocation. Together, these attacks provide a representative evaluation of lifecycle-oriented threats affecting Secure Runner execution.

6.1 Adversarial Modeling

The proposed specification represents adversarial behavior through explicit transition actions integrated into the global transition relation. Rather than introducing a separate attacker process or external threat model, hostile actions are modeled using the same state-transition semantics that govern legitimate system execution. Consequently, adversarial behavior becomes an integral part of the formal specification and is explored exhaustively during model checking.

Each adversarial transition captures a specific class of attacks identified in the threat model. These transitions operate over the same global state as trusted and lifecycle transitions, modifying only those variables that an attacker is assumed capable of influencing under the adopted security assumptions. This design preserves a unified execution semantics while avoiding the need for specialized attacker abstractions.

As an illustrative example, the transition below presents the transition modeling session replay. Unlike an external attacker model, replay is represented as a first-class state transition that directly updates the global authorization state. Once a previously observed session is replayed, the affected runner is immediately forced into a fail-closed state by revoking trust, invalidating execution capability, revoking the secret lease, and aborting execution.

```
ReplaySession(r, s) ==
  s \in attackerKnownSessions
```

```

/\ trustSessionId[r] = Null
/\ replayedSession' =
    [replayedSession EXCEPT ![r] = TRUE]
/\ trustState' =
    [trustState EXCEPT ![r] = "Revoked"]
/\ runnerState' =
    [runnerState EXCEPT ![r] = "Aborted"]
/\ executionCapability' =
    [executionCapability EXCEPT ![r] = FALSE]
/\ secretLeaseState' =
    [secretLeaseState EXCEPT ![r] = "Revoked"]
/\ revocationReason' =
    [revocationReason EXCEPT ![r] = "SessionReplay"]

```

This transition illustrates the modeling philosophy adopted throughout the specification. Rather than attempting to model network traffic or protocol exchanges, the attack is represented directly through its security-relevant effects on the global state. Consequently, replay becomes another transition explored non-deterministically by TLC alongside trusted execution, allowing replay resistance to be verified over every reachable execution trace rather than over predefined attack scenarios.

An important consequence of this modeling strategy is that adversarial and legitimate transitions compete nondeterministically throughout execution. During state-space exploration, replay attempts, stale authorization reuse, unauthorized policy manipulation, and other hostile actions may occur at any point in the execution lifecycle where their enabling conditions are satisfied. The verification process therefore considers not only the attacks themselves, but also every possible interleaving between hostile and legitimate execution.

This approach is particularly well suited to CI/CD infrastructures, where provisioning, attestation, authorization, workload execution, and revocation occur asynchronously across multiple distributed components. By allowing adversarial transitions to interleave freely with trusted execution, the specification captures realistic race conditions and lifecycle inconsistencies that would be difficult to analyze using sequential execution models.

The adversarial model currently incorporates five representative classes of attacks: replay of attestation evidence, replay of execution sessions, unauthorized policy injection, cross-runner secret reuse, and use-after-expiry execution. Although these attacks target different portions of the global state, they all attempt to violate one or more of the lifecycle-oriented security objectives introduced in Section 3. Consequently, each adversarial transition is associated with one or more formally verified safety and liveness properties that constrain its observable effects on the system.

Rather than demonstrating that individual attacks fail in isolation, the resulting specification establishes that every reachable execution—including arbitrary interleavings of trusted, lifecycle, and adversarial transitions—preserves the security guarantees verified by the model checker.

6.2 Replay-Oriented Attacks

Replay attacks represent one of the most fundamental threats to lifecycle-oriented authorization systems. Rather than compromising cryptographic primitives or trusted authorities, replay attacks attempt to reuse previously valid authorization artifacts after their intended security context has changed. Within ephemeral CI/CD environments, such attacks may exploit stale attestation evidence, expired execution sessions, or residual authorization state to recover privileges that should no longer exist.

The proposed specification models two complementary replay scenarios. The first concerns the replay of previously observed attestation evidence, while the second targets the reuse of stale execution sessions. Although both attacks attempt to exploit historical authorization state, they violate different security assumptions and therefore require distinct formal guarantees.

Evidence replay attempts to reuse attestation material generated during an earlier execution. Since trust establishment requires both freshness and correct binding to the originating runner and execution session, replayed evidence cannot satisfy the conditions necessary to activate trusted execution. This property is captured by the invariant:

```
Inv_NoTrustFromReplayedEvidence ==
  \A r \in RunnerIds :
    replayedEvidence[r] => trustState[r] # "Active"
```

The invariant guarantees that replayed attestation evidence cannot establish or preserve an active trust relationship. Consequently, previously valid attestation artifacts cannot be reused to bypass authorization checks or recover execution privileges.

Session replay targets a different stage of the lifecycle by attempting to reuse an execution session that has already expired or been revoked. Unlike evidence replay, which attacks trust establishment, session replay attempts to preserve execution capability after the corresponding authorization context is no longer valid.

The specification constrains this behavior through both safety and liveness properties. Safety invariants prevent replayed sessions from recovering authorization, while complementary liveness properties ensure that any replay attempt eventually progresses toward revocation and lifecycle termination. Consequently, stale execution state cannot indefinitely preserve privileged execution, even if replay attempts occur concurrently with legitimate lifecycle transitions.

Together, these replay-oriented properties establish authorization freshness throughout the complete execution lifecycle. Trust establishment remains bound to fresh attestation evidence, execution sessions remain temporally constrained, and replay-derived authorization cannot persist beyond the lifetime of its original security context.

6.3 Authorization and Policy Attacks

Beyond replay-oriented threats, the proposed specification models adversarial behaviors that attempt to modify or extend an existing authorization context after trust has already been established. These attacks target policy integrity, secret isolation, and authorization lifetime rather than the initial trust establishment process.

The first modeled attack concerns unauthorized policy injection. Once communication policies have been successfully evaluated and authorized, they become immutable throughout the execution lifecycle. An attacker may nevertheless attempt to expand the set of permitted communication endpoints in order to enable unauthorized network access or data exfiltration.

The specification prevents such behavior through invariants constraining policy evolution after authorization. Any attempt to modify an authorized policy immediately violates the corresponding safety properties and ultimately leads to fail-closed revocation. Consequently, communication privileges cannot be expanded after trusted execution has been established.

A second class of attacks targets secret isolation. Rather than attempting to obtain new secrets, an adversary attempts to reuse secret material issued to another runner instance. Since secret leases remain explicitly bound to both the originating runner and the corresponding trusted execution session, secret propagation across independent runners is prohibited throughout the execution lifecycle.

Finally, the specification models use-after-expiry execution attempts. This adversarial behavior assumes that an attacker attempts to continue execution after authorization has expired or trust has been revoked. Because execution capability is derived from the simultaneous validity of trust establishment, secret availability, and execution authorization, expiration of any one of these conditions immediately invalidates execution eligibility.

Unlike conventional authorization models based on persistent privileges, the proposed lifecycle-oriented model treats authorization as a continuously evolving security state. Consequently, policy mutation, secret reuse, and post-expiration execution cannot preserve privileged execution because every authorization decision remains continuously constrained by the verified safety and liveness properties governing the complete execution lifecycle.

Collectively, these attacks complement the replay-oriented scenarios discussed previously by demonstrating that authorization cannot be extended beyond its intended scope, lifetime, or execution context. Together, both categories provide comprehensive coverage of the lifecycle-oriented threats considered by the proposed Secure Runner model.

6.4 Verified Security Guarantees

The adversarial verification results demonstrate that every modeled attack is constrained by one or more formally verified correctness properties governing the authorization lifecycle. Rather than relying on informal security arguments,

each adversarial behavior is explicitly associated with the corresponding security objectives introduced in Section 3 and verified through exhaustive state-space exploration.

Table 7 summarizes this correspondence by relating each adversarial behavior to its targeted security objectives, the formally verified safety and liveness properties, and the resulting security guarantees.

Table 7. Relationship between modeled adversarial behaviors and formally verified guarantees.

Adversarial Behaviour	Objectives	Properties	Guarantee
Evidence play	Re- O1, O6	<i>Inv_Freshness</i> , <i>Inv_NoTrustFromReplayedEvidence</i>	Replayed attestation evidence cannot establish or preserve trusted execution.
Session Replay	O2, O5	<i>Inv_NoTrustFromReplayedSession</i> , <i>Inv_ReplayedSessionForcesAbort</i> , <i>SessionReplayEventuallyTerminated</i>	Stale execution sessions cannot recover execution capability and are eventually terminated.
Unauthorized Policy Injection	O4	<i>Inv_PolicyLockedImmutable</i> , <i>Inv_NoInjectedEgressAllowed</i>	Authorized communication policies remain immutable throughout execution.
Cross-Runner Secret Reuse	O3	<i>Inv_NoCrossRunnerSecretLeak</i> , <i>Inv_NoCrossRunnerSecretReuse</i>	Secret leases remain isolated and cannot propagate across independent runners.
Use-After-Expiry Execution	O2, O5	<i>Inv_NoUseAfterTrustExpiry</i> , <i>Inv_TrustDestroyed</i>	Expired authorization cannot preserve execution capability or trusted execution.

The correspondence presented in Table 7 highlights that every modeled adversarial behavior is constrained by multiple complementary correctness properties rather than by a single invariant. Safety properties prevent unauthorized security states from becoming reachable, whereas liveness properties ensure that adversarial executions cannot indefinitely preserve privileged state once authorization has been invalidated.

Collectively, these results demonstrate that replay-derived trust reuse, stale authorization persistence, unauthorized policy evolution, cross-runner secret propagation, and post-expiration execution are all prevented by the formal semantics of the proposed lifecycle model. Under the assumptions defined in Section 3, every reachable execution either preserves the required authorization constraints or is forced toward fail-closed revocation and lifecycle termination.

Consequently, the security guarantees established by the specification emerge from the interaction of independently verified trust, authorization, secret management, policy enforcement, and revocation semantics rather than from isolated protection mechanisms. This provides a rigorous formal basis for reasoning about hardened Secure Runner execution under adversarial conditions.

6.5 Residual Risks and Scope

As with any formal model, the guarantees established by the proposed specification must be interpreted within the assumptions adopted during modeling. The objective of this work is to verify the correctness of the authorization lifecycle governing Secure Runner execution rather than to model every aspect of a complete CI/CD infrastructure.

Accordingly, the specification intentionally abstracts away several classes of attacks and implementation details that lie outside the scope of lifecycle-oriented authorization verification. These include hardware side-channel attacks, vulnerabilities affecting Trusted Execution Environment implementations, cryptographic failures, denial-of-service attacks, workload semantic correctness, operating-system behavior, and deployment-specific performance characteristics. Such aspects require complementary analysis techniques, experimental validation, or implementation-level verification.

Table 8 summarizes the principal assumptions adopted by the formal model together with the rationale for excluding certain implementation-level concerns. These assumptions define the scope within which the verified security guarantees are valid.

Similarly, the trusted computing base defined in Section 3 assumes the correctness of the orchestration service, attestation authority, policy authority, secret management infrastructure, and the underlying cryptographic primitives. The specification therefore does not model compromise of trusted control-plane components or failures of the hardware root of trust itself.

These modeling decisions are intentional rather than limitations of the verification methodology. By abstracting implementation-specific details while preserving the security-relevant semantics of trust establishment, authorization, secret management, policy enforcement, and revocation, the model isolates the fundamental correctness properties governing lifecycle-oriented authorization. This level of abstraction enables exhaustive exploration of the complete reachable state space while keeping the verification problem tractable.

The formal guarantees established in this work should therefore be interpreted as correctness results for the proposed authorization model rather than

Table 8. Model scope and residual assumptions.

Out-of-Scope Aspect	Reason for Abstraction
TEE implementation vulnerabilities	vul- The model assumes correct Trusted Execution Environment behavior and verifies only the authorization semantics built upon it.
Cryptographic failures	primitive Cryptographic algorithms are treated as trusted primitives; only their security-relevant effects are modeled.
Operating-system and hypervisor internals	hy- Platform implementation details are abstracted to focus on lifecycle-oriented authorization correctness.
Denial-of-service attacks	Availability attacks are outside the scope of authorization verification and require complementary evaluation.
Application workload correctness	cor- CI/CD jobs are modeled as abstract workloads whose internal computation does not affect authorization semantics.
Performance and scalability	The specification verifies functional correctness rather than execution performance or resource utilization.
Compromise of trusted control-plane components	The orchestrator, attestation authority, policy authority, and secret manager constitute part of the trusted computing base assumed by the model.

for a specific Secure Runner implementation. Nevertheless, because every reachable execution satisfying the adopted assumptions has been explored exhaustively, the verified properties provide strong assurance that replay resistance, authorization freshness, secret isolation, policy immutability, and fail-closed revocation are intrinsic properties of the modeled lifecycle semantics.

Future work will progressively reduce the abstraction gap by complementing explicit-state verification with symbolic verification using Apalache, deductive proofs using TLAPS, and implementation-level validation of the Secure Runner architecture. Together, these complementary approaches will strengthen confidence that the formally verified lifecycle properties remain preserved in practical CI/CD deployments.

7 Evaluation

7.1 Experimental Setup

The objective of the evaluation is to determine whether the proposed Secure Runner specification preserves its security guarantees throughout the complete execution lifecycle under both benign and adversarial conditions. Rather than evaluating a concrete implementation, the experiments assess the correctness of the formal specification through exhaustive state-space exploration using the TLC explicit-state model checker.

The evaluation considers a representative abstract deployment consisting of two runner instances executing independent CI/CD jobs over finite domains for

runners, jobs, nonces, execution sessions, and authorized network endpoints. These domains preserve the security-relevant interactions required to reason about trust establishment, authorization, secret management, policy enforcement, and revocation while enabling exhaustive exploration of every reachable execution.

The selected domain sizes represent the smallest non-trivial configuration capable of exposing all security-relevant interactions modeled in this work. In particular, two runner instances are sufficient to capture cross-runner isolation and secret reuse attempts, while two jobs, sessions, nonces, and endpoints enable modeling of replay attacks, stale authorization reuse, policy conflicts, and lifecycle interactions between independent executions. Increasing these domains would significantly enlarge the explored state space without introducing fundamentally new classes of behaviors under the adopted abstraction.

The explored transition relation includes both trusted lifecycle operations and adversarial behavior, allowing TLC to evaluate arbitrary interleavings involving replay attacks, stale session reuse, unauthorized policy injection, cross-runner secret reuse, and use-after-expiry execution attempts. In total, the specification was verified against 38 safety invariants and 9 liveness properties. To analyze the impact of adversarial behavior and temporal reasoning on verification complexity, four complementary verification configurations were evaluated while preserving the same underlying formal specification.

Experimental Environment. All verification experiments were performed using the TLC explicit-state model checker on the Deucalion high-performance computing (HPC) system. Verification jobs were submitted through the Slurm workload manager and executed on the `normal-arm` partition using a single compute node with four CPU cores and 28 GB of allocated memory.

TLC was executed using OpenJDK 17 with a maximum Java heap size of 24 GB and four parallel worker threads. Disk-based fingerprint storage (`MSBDiskFPSet`) and the disk-backed state queue (`DiskStateQueue`) were enabled to support exploration of the large reachable state space while remaining within the available memory budget.

The complete verification artifacts, including the TLA+ specification, TLC configuration files, execution scripts, and reproducibility instructions, are publicly available in the companion artifact repository (Section 8).

The experimental environment is summarized in Table 9.

Table 10 summarizes the statistics obtained for the largest verification campaign.

7.2 Verification Configuration

To evaluate different aspects of the proposed specification, the verification campaign was organized into four complementary configurations. Rather than executing a single exhaustive verification, each configuration isolates a particular

Table 9. Experimental environment used for TLC verification.

Component	Configuration
Model checker	TLC 2.20 explicit-state model checker
Execution platform	Deucalion HPC system
Workload manager	Slurm
Partition	normal-arm
Compute resources	1 node, 4 CPU cores, 28 GB RAM
Java runtime	OpenJDK 17
Maximum Java heap	24 GB
Parallel workers	4
Fingerprint set	MSBDiskFPSet
State queue	DiskStateQueue
Specification size (TLA+ LOC)	1,487

Table 10. Summary of the largest verification campaign (LivenessSpec full).

Metric	Value
States generated	84,869,530
Distinct states	15,414,082
State graph depth	35
Maximum outdegree	16
Temporal branches	18
State-space exploration	18 h 37 min
Temporal verification	9 h 22 min
Total verification time	1 day 3 h
Verification result	All properties satisfied

dimension of the model, allowing the impact of adversarial behavior and temporal verification to be analyzed independently.

Table 11 summarizes the evaluated configurations.

The **Spec** configuration establishes the baseline verification of the complete adversarial specification by checking all safety properties without fairness assumptions. This configuration isolates the cost of verifying the safety invariants over the full state space while excluding temporal reasoning.

The **LivenessWithoutAdv** configuration introduces fairness assumptions while removing adversarial transitions. Its purpose is to evaluate the temporal behavior of the trusted execution lifecycle independently of hostile actions, thereby providing a reference point for understanding the additional verification effort introduced by adversarial behavior.

The **LivenessSpec (small)** configuration enables both adversarial transitions and liveness verification over a reduced domain consisting of a single runner,

Table 11. Verification configurations evaluated with TLC (R: runners, J: jobs, N: nonces, S: sessions, E: endpoints).

Configuration	Domain Size	Adversary	Fairness	Verified Properties
Spec	2R/2J/2N/2S/2E	Yes	No	38 Safety
LivenessWithoutAdv	2R/2J/2N/2S/2E	No	Yes	38 Safety, 9 Liveness
LivenessSpec (small)	1R/1J/1N/1S/1E	Yes	Yes	38 Safety, 9 Liveness
LivenessSpec (full)	2R/2J/2N/2S/2E	Yes	Yes	38 Safety, 9 Liveness

job, nonce, session, and endpoint. This configuration serves primarily as a sanity check for the complete temporal specification, allowing rapid validation of all safety and liveness properties before exploring the substantially larger state space of the full model.

Finally, the **LivenessSpec (full)** configuration represents the complete verification campaign reported in this work. It combines the full adversarial model, fairness assumptions, and the complete set of safety and liveness properties over the largest explored domain. This configuration provides the strongest evidence that the proposed lifecycle-oriented authorization model preserves its security guarantees throughout all reachable executions considered by the specification.

Together, these four configurations progressively increase the complexity of the verification problem while enabling direct comparison of the effects of fairness assumptions, adversarial behavior, and domain size on state-space exploration.

7.3 Verification Results

Table 12 reports the state-space exploration statistics obtained for the four verification configurations.

Table 12. Verification results obtained with TLC. Execution times are reported in HH:MM:SS format.

Configuration	Generated	Distinct	Runtime	Result
Spec	84,869,530	15,414,082	00:23:42	No violations
LivenessWithoutAdversary	639,834	229,970	01:09:00	No violations
LivenessSpec (small)	659	255	00:00:08	No violations
LivenessSpec (full)	84,869,530	15,414,082	27:40:00	No violations

All verification campaigns completed successfully without identifying violations of the specified safety invariants or liveness properties. Across all explored executions, the specification consistently preserves authorization correctness, attestation-bound trust establishment, policy immutability, secret isolation, and fail-closed revocation.

The largest safety verification explores more than 84 million generated states and over 15 million distinct states while including adversarial transitions directly in the state space. This demonstrates that replay attacks, stale authorization

reuse, unauthorized policy modification, cross-runner secret reuse, and use-after-expiry execution cannot produce reachable states violating the modeled security invariants.

Temporal verification introduces the dominant computational cost. Although the reachable state space remains unchanged, checking the nine liveness properties requires TLC to analyze 18 temporal branches covering more than 277 million graph states. Consequently, total verification time increases from approximately 24 minutes for safety verification to more than one day for the complete verification campaign with the **LivenessSpec (full)** configuration.

The reduced-domain configuration, **LivenessSpec (small)**, provides an important validation of the complete temporal specification. By executing the full adversarial model over a minimal state space, it enables rapid confirmation that both safety and liveness properties are correctly specified before performing the considerably more expensive verification of the complete model.

Similarly, the **LivenessWithoutAdv** configuration isolates the impact of fairness assumptions independently of adversarial behavior. Comparison with the full adversarial configuration demonstrates that both adversarial transitions and temporal reasoning contribute significantly to verification complexity while preserving the correctness guarantees established by the specification.

Overall, the experimental results demonstrate that the proposed lifecycle-oriented authorization model remains internally consistent under exhaustive explicit-state exploration. Despite incorporating adversarial behavior directly into the transition relation and verifying both safety and temporal properties, TLC successfully explored every reachable execution within the specified domains without identifying counterexamples.

The reduced-domain and adversary-free configurations demonstrate the usefulness of incremental verification during model development. Smaller configurations enable rapid validation of newly introduced safety and temporal properties before executing the substantially more expensive verification over the complete adversarial state space. This workflow proved particularly valuable during the iterative refinement of the specification while preserving identical verification semantics.

7.4 Discussion

The verification results indicate that lifecycle-oriented authorization provides a suitable abstraction for reasoning about Secure Runner security. Rather than modeling low-level implementation mechanisms, the specification captures the evolution of trust, execution capability, policy state, and secret availability throughout the execution lifecycle. The successful verification of all specified safety and liveness properties suggests that these abstractions are sufficient to express the security guarantees targeted by the proposed architecture.

A second observation concerns the explicit integration of adversarial behavior into the transition system. Instead of verifying individual attack scenarios independently, the model incorporates replay attacks, stale authorization reuse,

policy injection, cross-runner secret reuse, and use-after-expiry attempts as ordinary transitions. Consequently, the verified guarantees hold for every reachable execution under arbitrary interleavings of trusted and adversarial behavior within the assumptions of the model. This provides stronger assurance than validating isolated attack traces or manually constructed adversarial scenarios.

The evaluation also highlights the different computational characteristics of safety and temporal verification. Safety checking scales efficiently even for the complete adversarial specification, whereas liveness verification dominates the overall verification cost due to the additional graph exploration required to establish temporal properties under fairness assumptions. This behavior is consistent with the known characteristics of explicit-state model checking and suggests that safety verification can be performed frequently during iterative model development, while comprehensive temporal verification is better suited to milestone validation.

Finally, the use of multiple verification configurations proved valuable during the development of the specification. Smaller configurations enabled rapid validation of newly introduced invariants and temporal properties before executing the substantially more expensive verification of the complete adversarial model. Beyond improving development efficiency, this incremental workflow increases confidence that the verified guarantees are preserved consistently as verification complexity increases.

Overall, the evaluation demonstrates that exhaustive explicit-state verification can provide strong assurance for lifecycle-oriented authorization models of Secure Runner architectures while remaining computationally practical for representative deployment configurations.

7.5 Threats to Validity

The evaluation results presented in this work should be interpreted within the assumptions of the proposed formal model. As with any formal verification effort, the established guarantees apply to the specified system abstraction rather than to every possible implementation of a Secure Runner architecture.

Model Validity. The specification intentionally abstracts away implementation-specific details while preserving the security semantics required to reason about lifecycle-oriented authorization. Consequently, operating-system behavior, container runtimes, networking protocols, hardware implementation details, and workload semantics are not explicitly modeled. This level of abstraction is necessary to enable exhaustive state-space exploration while focusing on the correctness of authorization evolution.

Internal Validity. The verification assumes the correctness of the trusted computing base introduced in Section 3, including the orchestration service, attestation authority, policy authority, secret management infrastructure, and the underlying cryptographic primitives. Likewise, hardware compromise, cryptographic failures, and vulnerabilities affecting remote-attestation mechanisms are considered outside the scope of the model.

External Validity. The verification campaigns were conducted over finite domains comprising two runners, two jobs, two nonces, two execution sessions, and two network endpoints. Although these domains are sufficient to expose the targeted lifecycle interactions and enable exhaustive exploration, larger deployments may produce substantially larger state spaces requiring complementary verification techniques such as symbolic model checking or theorem proving.

Construct Validity. The reported results establish the correctness of the formal specification rather than the security of a concrete Secure Runner implementation. Consequently, the verified properties demonstrate that the proposed authorization model satisfies its specified safety and liveness requirements under the adopted assumptions, but they do not eliminate the need for implementation-level validation, performance evaluation, or security testing in operational CI/CD environments.

Despite these limitations, exhaustive exploration of every reachable execution provides strong evidence that replay resistance, authorization freshness, secret isolation, policy immutability, and fail-closed revocation are intrinsic properties of the proposed lifecycle-oriented authorization model. Future work will extend these results through symbolic verification with Apalache, deductive proofs with TLAPS, and experimental validation of the corresponding Secure Runner implementation.

Furthermore, exhaustive explicit-state model checking is inherently affected by state-space explosion. Although the selected abstractions enable complete exploration of all reachable executions within the chosen domains, verifying substantially larger deployments would require complementary techniques such as symbolic model checking or deductive verification.

8 Reproducibility

To facilitate independent verification of the reported results, all artifacts required to reproduce the verification campaign are publicly available.

The artifact repository contains the complete TLA+ specification, TLC configuration files, execution scripts, verification logs, experiment documentation, and the auxiliary material required to reproduce the evaluation presented in Section 7. Reproducing the reported experiments requires only the publicly available TLC model checker, a Java 17 runtime, and the provided execution scripts.

The artifact repository is available at:

<https://github.com/pduarte21/secure-runner-formal>

The verification results reported in this paper correspond to the immutable release **v1.0.0**, which preserves the exact specification, TLC configurations, execution scripts, and verification artifacts used during the evaluation:

<https://github.com/pduarte21/secure-runner-formal/releases/tag/v1.0.0>

Version-controlled releases ensure that future revisions of the specification remain distinguishable from the artifacts evaluated in this work while preserving the reproducibility of previously published results.

The repository is intended to evolve alongside the Secure Runner architecture. Future releases will extend the current verification artifacts with symbolic verification, deductive proofs, and additional implementation-oriented validation while maintaining versioned artifacts for each published verification campaign.

9 Related Work

9.1 CI/CD and Software Supply-Chain Security

The increasing sophistication of software supply-chain attacks has placed Continuous Integration and Continuous Deployment (CI/CD) infrastructures among the most security-critical components of modern software engineering. Recent empirical studies demonstrate that CI/CD pipelines expose numerous attack surfaces that can be exploited to compromise downstream software ecosystems at scale [29]. CI/CD pipelines routinely perform privileged operations such as source-code compilation, dependency resolution, artifact generation, software signing, secret consumption, and automated deployment. Consequently, compromising the execution environment of a CI/CD runner may allow attackers to manipulate software artifacts, exfiltrate credentials, or distribute malicious software to downstream users.

A substantial body of research has therefore focused on strengthening software supply-chain integrity through artifact provenance, traceability, and reproducible builds. Frameworks such as SLSA and in-toto establish mechanisms for generating verifiable provenance metadata, recording build steps, and protecting artifact integrity throughout the software delivery lifecycle [28, 38]. More broadly, recent systematizations of software supply-chain security identify build infrastructures, dependency ecosystems, development environments, and CI/CD platforms as some of the principal attack surfaces requiring stronger protection [39, 27].

Complementary research has investigated the security of CI/CD platforms themselves. Benedetti et al. demonstrated that insecure GitHub Actions workflows frequently expose excessive permissions and unsafe execution practices that can be exploited by attackers [6]. Similarly, workload identity frameworks such as SPIFFE and Zero Trust architectures strengthen authentication and authorization across software delivery pipelines while reducing reliance on long-lived credentials [4, 35]. Finally, industrial guidance increasingly recommends ephemeral self-hosted runners to eliminate residual execution state, reduce build contamination, and limit the impact of runner compromise [15, 2].

Collectively, these approaches significantly improve software provenance, workflow integrity, identity management, and operational hardening of CI/CD infrastructures. However, they primarily secure individual components of the software delivery process rather than the evolution of authorization during execution itself. Existing approaches typically assume that authorization remains valid once established and therefore do not formally model authorization freshness, replay resistance, revocation semantics, or the continuous validity of trust throughout

runner execution. The Secure Runner model proposed in this paper addresses this gap by treating authorization as a lifecycle property that is formally verified under both benign and adversarial execution.

9.2 Trusted Execution and Confidential Computing

Trusted Execution Environments (TEEs) and confidential computing technologies have become fundamental mechanisms for protecting sensitive workloads executing on partially trusted infrastructures. By combining hardware-enforced isolation with cryptographic remote attestation, these technologies enable external parties to verify the integrity and authenticity of an execution environment before releasing sensitive information. Consequently, confidential computing has emerged as one of the principal architectural foundations for protecting cloud-native services, confidential containers, AI workloads, and security-sensitive CI/CD infrastructures [24, 11, 13].

Intel SGX pioneered enclave-based trusted execution by providing isolated application enclaves together with hardware-supported remote attestation [24, 11]. More recently, AMD SEV-SNP and Intel TDX extended these concepts from process-level isolation to confidential virtual machines, allowing entire guest operating systems to execute inside hardware-protected environments with minimal software modification [1, 19]. These technologies now constitute the basis of confidential computing services offered by major cloud providers and have enabled practical deployment of confidential virtual machines, confidential containers, and attestation-aware cloud workloads [25, 18, 10].

A substantial body of research has investigated both the security assumptions and the practical deployment of confidential computing. Recent surveys compare different TEE architectures, trust models, attack surfaces, and implementation trade-offs across SGX, SEV-SNP, TDX, and related technologies [34, 13]. Other work has evaluated the performance and security characteristics of confidential virtual machines in production cloud environments [21]. Complementary efforts have focused on integrating remote attestation with workload identity, secure secret provisioning, and cloud key management systems, enabling cryptographic material to be released only after successful attestation of the executing workload [35, 33, 18].

These contributions provide the hardware mechanisms required to establish trusted execution environments. However, they primarily address *where* execution occurs rather than *how trust and authorization evolve* once execution has begun. In most confidential-computing architectures, remote attestation is treated as a prerequisite for releasing secrets, after which the workload is implicitly assumed to remain trustworthy throughout execution. Consequently, questions such as authorization freshness, execution capability ownership, policy immutability, revocation propagation, and replay-resistant execution semantics remain largely outside the scope of existing confidential-computing frameworks.

The Secure Runner model proposed in this paper complements rather than replaces confidential-computing technologies. TEEs provide the trusted execution substrate upon which the proposed architecture operates, while the formal

specification defines the lifecycle-oriented authorization semantics governing that execution. Instead of modeling trust as a one-time outcome of successful attestation, authorization is treated as a continuously evolving security property whose validity depends on attestation freshness, immutable policy enforcement, execution capability ownership, secret lease validity, and fail-closed revocation throughout the complete execution lifecycle.

9.3 Formal Methods for Secure Systems

Formal methods have become an established approach for providing mathematically rigorous correctness guarantees in safety- and security-critical systems. By modeling system behavior as a state-transition system or logical specification, they enable reasoning over all possible executions and allow subtle design flaws to be identified before implementation.

Among the various formal specification languages, TLA+ has become widely adopted for modeling concurrent and distributed systems. Its state-transition semantics naturally capture asynchronous execution, non-determinism, and temporal evolution, while the TLC model checker enables exhaustive exploration of finite state spaces against both safety and liveness properties [22]. The practical applicability of TLA+ has been demonstrated in numerous industrial systems, most notably within Amazon Web Services, where formal specifications have been used to validate the correctness of distributed cloud services before implementation [26].

Beyond the specification of distributed systems, formal methods have been extensively applied to security protocols, authentication mechanisms, access-control policies, and verified software. Symbolic verification frameworks such as ProVerif and Tamarin have become widely adopted for analyzing authentication and cryptographic protocols under powerful adversarial models [7, 5], while interactive theorem provers including Coq and Isabelle/HOL have enabled machine-checked proofs of security-critical software and protocol correctness [3, 14]. Languages such as F* further support the development of verified cryptographic software, whereas lightweight relational modeling frameworks such as Alloy facilitate early validation of structural and architectural properties [31, 23]. Collectively, these approaches demonstrate the effectiveness of mathematically rigorous verification across multiple layers of secure system design.

Despite this rich ecosystem of formal verification techniques, comparatively little work has focused on formally specifying and verifying the execution semantics of CI/CD infrastructures themselves. Existing formally verified security models predominantly target communication protocols, cryptographic exchanges, or access-control decisions, where authorization is typically represented as a static property established during authentication or attestation. Consequently, comparatively little attention has been devoted to modeling how trust, authorization, secret validity, policy enforcement, execution capability ownership, and revocation evolve throughout the lifetime of an ephemeral CI/CD execution environment.

The work presented in this paper extends the application of formal methods to secure software delivery infrastructures by modeling Secure Runner execution itself as a lifecycle-oriented authorization system. Rather than verifying isolated protocol exchanges, the proposed specification formally captures the evolution of authorization throughout execution and establishes safety and liveness guarantees for attestation-bound trust establishment, execution capability management, ephemeral secret leasing, immutable policy enforcement, replay resistance, and fail-closed revocation under explicit adversarial behavior.

9.4 Positioning of This Work

The preceding discussion shows that existing research addresses several complementary aspects of secure CI/CD infrastructures. Software supply-chain frameworks strengthen provenance and artifact integrity, confidential computing technologies provide hardware-assisted trusted execution, and formal methods establish correctness guarantees for distributed systems and security protocols. However, these research directions have largely evolved independently and therefore provide only partial solutions to the problem of securing ephemeral CI/CD execution.

Table 13 qualitatively compares representative research directions with respect to the design dimensions explicitly addressed by the proposed model. The comparison reflects the primary focus of each category rather than the complete feature set of individual systems.

Table 13. Comparison with representative categories of related work.

Category	Prov.	Attest.	Replay Resistance	Life.	Formal
SLSA / in-toto	Yes	No	No	No	No
CI/CD Security Analyses	Partial	No	Partial	No	No
Confidential Computing	No	Yes	Partial	No	No
Formal Security Protocols	Partial	No	Yes	Partial	Yes
Proposed Model	Yes	Yes	Yes	Yes	Yes

Prov.=Provenance; Attest.=Remote Attestation; Life.=Lifecycle-oriented Authorization; Formal=Formal Verification.

The comparison illustrates that existing approaches provide complementary rather than overlapping guarantees. Supply-chain security frameworks primarily strengthen software provenance and artifact integrity, confidential-computing technologies establish trusted execution environments, and formally verified security models focus on protocol correctness and communication semantics. However, none of these research directions explicitly specifies authorization as a lifecycle property whose correctness depends on the continuous interaction between attestation freshness, policy integrity, execution capability ownership, secret leasing, and revocation.

Rather than proposing another secure execution mechanism, the proposed Secure Runner model defines the formal authorization semantics governing secure execution. Unlike existing approaches, which typically verify isolated mechanisms such as provenance generation, trusted execution, or protocol correctness, the proposed specification unifies artifact provenance verification, attestation-bound trust establishment, ephemeral secret leasing, immutable policy enforcement, replay-aware authorization, and fail-closed revocation within a single formally verified lifecycle model. Consequently, the verified security guarantees arise not only from the correctness of individual mechanisms, but also from their formally verified interaction throughout the complete execution lifecycle of an ephemeral CI/CD runner.

To the best of our knowledge, this is the first formal specification of Secure Runner execution that jointly models attestation-bound trust establishment, replay-aware authorization, ephemeral secret leasing, immutable policy enforcement, lifecycle-oriented execution semantics, and explicit adversarial verification within the context of CI/CD runner execution.

10 Conclusion

This paper presented a formally verified lifecycle-oriented authorization model for hardened ephemeral CI/CD runner execution. The proposed specification addresses an important gap in existing CI/CD security research, which typically establish trust only during the initial stages of execution and subsequently assume that authorization remains valid throughout the lifetime of the runner. In contrast, the proposed model treats authorization as a dynamic security property whose validity evolves continuously according to trust establishment, execution capability ownership, secret lease validity, immutable policy enforcement, and revocation semantics.

The proposed Secure Runner authorization model was formalized as a distributed state-transition system using TLA+, enabling precise specification of both trusted execution and explicitly modeled adversarial behavior. Unlike conventional security models that verify isolated protocol exchanges or access-control decisions, the proposed specification captures the complete execution lifecycle of ephemeral CI/CD runners, including provisioning, attestation, authorization, execution, revocation, and teardown.

The specification was evaluated through exhaustive explicit-state model checking using TLC. Across all verification configurations, the model satisfied all 38 safety properties and 9 liveness properties without producing counterexamples, providing evidence that the proposed lifecycle-oriented authorization semantics preserve authorization freshness, replay resistance, policy integrity, secret isolation, and fail-closed revocation throughout every reachable execution considered by the model.

Beyond Secure Runners themselves, this work demonstrates that lifecycle-oriented authorization constitutes a useful abstraction for reasoning about trusted execution in modern software delivery systems. Modeling authorization as a

continuously evolving state, rather than as a one-time authentication decision, enables security guarantees that naturally capture trust evolution, revocation, secret validity, and execution capability throughout the lifetime of a workload.

To the best of our knowledge, this is the first formal specification of a lifecycle-oriented authorization model for ephemeral CI/CD runner execution that jointly models attestation-bound trust establishment, replay-aware authorization, ephemeral secret leasing, immutable policy enforcement, and explicit adversarial behavior. As such, it provides a rigorous formal foundation upon which future Secure Runner implementations and formally verified CI/CD security architectures can be systematically developed.

11 Future Work

Although the proposed model establishes strong guarantees for the lifecycle-oriented authorization semantics considered in this work, it also opens several opportunities for extending both the specification and its practical application.

A first direction concerns the continued evolution of the adversarial model. Future versions of the specification may extend the adversarial model to capture additional attack classes relevant to secure CI/CD infrastructures, including control-plane compromise, multi-stage workflow interactions, delegated trust relationships, richer secret propagation scenarios, and more sophisticated authorization abuse strategies. Extending the adversarial model will enable increasingly realistic execution environments to be analyzed while preserving exhaustive formal reasoning.

A second direction focuses on complementary verification techniques. While TLC provides exhaustive explicit-state verification over finite domains, future work will integrate symbolic model checking using Apalache together with deductive verification using TLAPS. Combining explicit-state exploration, symbolic verification, and machine-checked proofs will increase confidence that the verified properties remain valid under broader execution assumptions and larger state spaces.

A third direction concerns the realization of the formally verified model within the BringTrust Secure Runner architecture. Future work will investigate how lifecycle-oriented authorization semantics can be preserved throughout a production-ready implementation integrating confidential computing, remote attestation, policy enforcement, ephemeral secret provisioning, runtime monitoring, and hardware-backed trust anchors.

More broadly, this work demonstrates that authorization can be treated as a first-class temporal property of CI/CD execution rather than as a static access-control decision. This perspective is not limited to Secure Runners and may provide a general formal foundation for reasoning about trust evolution across software supply-chain infrastructures, including workflow orchestration, artifact distribution, deployment pipelines, confidential execution platforms, and other security-critical cloud-native systems. Extending lifecycle-oriented authorization to these broader domains represents a promising direction for future research.

Acknowledgements

This work was partially supported by the BringTrust project (ALGARVE-FEDER-01489500), by NOVA LINES (UID/04516/NOVA Laboratory for Computer Science and Informatics) with financial support from FCT, I.P. The authors also acknowledge the computational resources provided by the Deucalion supercomputer (granted by the EuroCC 2 Portugal(European High-Performance Computing Joint Undertaking under Grant Agreement No. 101101903). which were used to perform the model-checking experiments reported in this paper.

References

1. Advanced Micro Devices: Amd sev-snp: Strengthening vm isolation with integrity protection and more. <https://docs.amd.com/v/u/en-US/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more> (2020)
2. Amazon Web Services: Best practices working with self-hosted github action runners at scale on aws. <https://aws.amazon.com/blogs/devops/best-practices-working-with-self-hosted-github-action-runners-at-scale-on-aws/> (2024)
3. Appel, A.W.: Coq’s vibrant ecosystem for verification engineering (invited talk). In: Proceedings of the 11th ACM SIGPLAN International Conference on Certified Programs and Proofs. p. 2–11. CPP 2022, Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3497775.3503951>, <https://doi.org/10.1145/3497775.3503951>
4. Avirneni, S.T.: Establishing workload identity for zero trust ci/cd: From secrets to spiffe-based authentication (2025), <https://arxiv.org/abs/2504.14760>
5. Basin, D., Cremers, C., Dreier, J., Sasse, R.: Symbolically analyzing security protocols using tamarin. ACM SIGLOG News **4**(4), 19–30 (Nov 2017). <https://doi.org/10.1145/3157831.3157835>, <https://doi.org/10.1145/3157831.3157835>
6. Benedetti, G., Verderame, L., Merlo, A.: Automatic security assessment of github actions workflows. In: Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses. p. 37–45. CCS ’22, ACM (Nov 2022). <https://doi.org/10.1145/3560835.3564554>, <http://dx.doi.org/10.1145/3560835.3564554>
7. Blanchet, B.: Modeling and verifying security protocols with the applied pi calculus and proverif. Found. Trends Priv. Secur. **1**(1–2), 1–135 (Oct 2016). <https://doi.org/10.1561/33000000004>, <https://doi.org/10.1561/33000000004>
8. Booth, H., Ogata, M., Kent, K., Souppaya, M., Dodson, D.: Secure software development framework (ssdf) version 1.2: Recommendations for mitigating the risk of software vulnerabilities. Tech. Rep. SP 800-218 Rev. 1 (Initial Public Draft), National Institute of Standards and Technology (NIST) (2025). <https://doi.org/10.6028/NIST.SP.800-218r1.ipd>, <https://csrc.nist.gov/pubs/sp/800/218/r1/ipd>
9. Codecov: Security update. <https://about.codecov.io/security-update/> (2021)
10. Confidential Computing Consortium: Does Confidential Computing Work with Containers? <https://confidentialcomputing.io/2025/03/06/does-confidential-computing-work-with-containers/> (Mar 2025), accessed: 2026-07-03
11. Costan, V., Devadas, S.: Intel sgx explained (2016), <https://eprint.iacr.org/2016/086.pdf>

12. Cybersecurity and Infrastructure Security Agency: Advanced persistent threat compromise of government agencies, critical infrastructure, and private sector organizations. <https://www.cisa.gov/news-events/cybersecurity-advisories/aa20-352a> (2020)
13. Feng, D., Qin, Y., Feng, W., Li, W., Shang, K., Ma, H.: Survey of research on confidential computing. *IET Communications* **18**(9), 535–556 (Apr 2024). <https://doi.org/10.1049/cmu2.12759>, <https://doi.org/10.1049/cmu2.12759>
14. From, A.H.: An isabelle/hol framework for synthetic completeness proofs. In: *Proceedings of the 14th ACM SIGPLAN International Conference on Certified Programs and Proofs*. p. 171–186. CPP '25, Association for Computing Machinery, New York, NY, USA (2025). <https://doi.org/10.1145/3703595.3705882>, <https://doi.org/10.1145/3703595.3705882>
15. GitHub: Self-hosted runners reference. <https://docs.github.com/en/actions/reference/runners/self-hosted-runners> (2025)
16. GitHub: Understanding github actions. <https://docs.github.com/en/actions/get-started/understand-github-actions> (nd)
17. GitLab: Gitlab ci/cd runners. <https://docs.gitlab.com/ci/runners/> (2026)
18. Google Cloud: Confidential Computing. <https://cloud.google.com/confidential-computing> (2025)
19. Intel Corporation: Intel® trust domain extensions (intel® tdx). <https://cdrdv2.intel.com/v1/dl/getContent/690419> (2025)
20. Jeff Johnson, Fred Plan, Adrian Sanchez, Renato Fontana, Jake Nicastro, Dimitar Andonov, Marius Fodoreanu, Daniel Scott: 3cx software supply chain compromise initiated by a previous software supply chain compromise (2023), <https://cloud.google.com/blog/topics/threat-intelligence/3cx-software-supply-chain-compromise>
21. Kollenda, K.: General overview of amd sev-snp and intel tdx. Friedrich-Alexander-Universität Erlangen-Nürnberg, Technical Report (2022)
22. Lamport, L.: *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley (2002)
23. Macedo, N., Cunha, A.: Automatic unbounded verification of alloy specifications with prover9 (2012), <https://arxiv.org/abs/1209.5773>
24. McKeen, F., Alexandrovich, I., Berenzon, A., Rozas, C.V., Shafi, H., Shanbhogue, V., Savagaonkar, U.R.: Innovative instructions and software model for isolated execution. In: *Proceedings of the 2nd International Workshop on Hardware and Architectural Support for Security and Privacy*. HASP '13, Association for Computing Machinery, New York, NY, USA (2013). <https://doi.org/10.1145/2487726.2488368>, <https://doi.org/10.1145/2487726.2488368>
25. Misono, M., Stavrakakis, D., Santos, N., Bhatotia, P.: Confidential vms explained: An empirical analysis of amd sev-snp and intel tdx. *Proc. ACM Meas. Anal. Comput. Syst.* **8**(3) (Dec 2024). <https://doi.org/10.1145/3700418>, <https://doi.org/10.1145/3700418>
26. Newcombe, C., Rath, T., Zhang, F., Munteanu, B., Brooker, M., Deardeuff, M.: How amazon web services uses formal methods. *Commun. ACM* **58**(4), 66–73 (Mar 2015). <https://doi.org/10.1145/2699417>, <https://doi.org/10.1145/2699417>
27. Okafor, C., Schorlemmer, T.R., Torres-Arias, S., Davis, J.C.: Sok: Analysis of software supply chain security by establishing secure design properties (2024), <https://arxiv.org/abs/2406.10109>
28. OpenSSF SLSA Community: About slsa (2025), <https://slsa.dev/spec/v1.2/about>

29. Pan, Z., Shen, W., Wang, X., Yang, Y., Chang, R., Liu, Y., Liu, C., Liu, Y., Ren, K.: Ambush from all sides: Understanding security threats in open-source software ci/cd pipelines. *IEEE Trans. Dependable Secur. Comput.* **21**(1), 403–418 (Jan 2024). <https://doi.org/10.1109/TDSC.2023.3253572>, <https://doi.org/10.1109/TDSC.2023.3253572>
30. Przymus, P., Durieux, T.: Wolves in the repository: A software engineering analysis of the xz utils supply chain attack (2025), <https://arxiv.org/abs/2504.17473>
31. Rastogi, A.: Proof-oriented programming for high-assurance systems. In: *Proceedings of the 16th Innovations in Software Engineering Conference. ISEC '23*, Association for Computing Machinery, New York, NY, USA (2023). <https://doi.org/10.1145/3578527.3581769>, <https://doi.org/10.1145/3578527.3581769>
32. Rose, S., Borchert, O., Mitchell, S., Connelly, S.: Zero trust architecture. *Tech. Rep. NIST Special Publication 800-207*, National Institute of Standards and Technology (2020). <https://doi.org/10.6028/NIST.SP.800-207>, <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-207.pdf>
33. Schear, N., Cable, P.T., Moyer, T.M., Richard, B., Rudd, R.: Bootstrapping and maintaining trust in the cloud. In: *Proceedings of the 32nd Annual Conference on Computer Security Applications*. p. 65–77. *ACSAC '16*, Association for Computing Machinery, New York, NY, USA (2016). <https://doi.org/10.1145/2991079.2991104>, <https://doi.org/10.1145/2991079.2991104>
34. Schneider, M., Masti, R.J., Shinde, S., Capkun, S., Perez, R.: Sok: Hardware-supported trusted execution environments (2022), <https://arxiv.org/abs/2205.12742>
35. SPIFFE Project: Spiffe: Secure production identity framework for everyone. <https://spiffe.io> (2025)
36. StepSecurity: Harden-runner documentation (2026), <https://docs.stepsecurity.io/github-actions/harden-runner>
37. Sysdig Threat Research Team: Security mechanism bypass in harden-runner github action (2025), <https://www.sysdig.com/blog/security-mechanism-bypass-in-harden-runner-github-action>
38. Torres-Arias, S., Afzali, H., Kuppusamy, T.K., Curtmola, R., Cappos, J.: in-toto: Providing farm-to-table guarantees for bits and bytes. In: *28th USENIX Security Symposium (USENIX Security 19)*. pp. 1393–1410. *USENIX Association*, Santa Clara, CA (Aug 2019), <https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias>
39. Williams, L., Benedetti, G., Hamer, S., Paramitha, R., Rahman, I., Tamanna, M., Tystahl, G., Zahan, N., Morrison, P., Acar, Y., Cukier, M., Kästner, C., Kapravelos, A., Wermke, D., Enck, W.: Research directions in software supply chain security. *ACM Trans. Softw. Eng. Methodol.* **34**(5) (May 2025). <https://doi.org/10.1145/3714464>, <https://doi.org/10.1145/3714464>